# OPERATING SYSTEMS LABORATORY
# RECORD

Name : S.Kaviya

Register No. : 241701023

| Exp. | Title |
|---|---|
| 1A | Shell script – arithmetic operations using expr |
| 1B | Shell script – leap year check using if-elif-else |
| 2A | Shell script – reverse a number using while loop |
| 2B | Shell script – Fibonacci series using for loop |
| 3 | System calls – fork, exec, getpid, opendir, readdir |
| 4 | CPU scheduling – FCFS, SJF, Priority, Round Robin |
| 5 | IPC using shared memory |
| 6 | Producer–Consumer problem using semaphores |
| 7 | Banker's algorithm – deadlock avoidance |
| 8 | Contiguous memory allocation – Best Fit |
| 9 | Contiguous memory allocation – First Fit |
| 10 | Page replacement – FIFO |
| 11 | Page replacement – LRU |
| 12 | Page replacement – Optimal |

# EXPERIMENT 1A

## SHELL SCRIPT – ARITHMETIC OPERATIONS USING expr COMMAND

**AIM:** To perform basic arithmetic operations (addition, subtraction, multiplication, division and remainder) using the expr command in a shell script.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, Terminal, Bash shell.

**PROCEDURE:**

1. Open the Fedora terminal in Oracle VirtualBox.

2. Create the script using nano arithmetic.sh.

3. Type the script and save the file.

4. Make it executable using chmod +x arithmetic.sh.

5. Run it using ./arithmetic.sh and enter two integers.

**PROGRAM:**

```
echo "Enter two integers:"
read x y
echo "Sum: $(expr "$x" + "$y")"
echo "Difference: $(expr "$x" - "$y")"
echo "Product: $(expr "$x" \* "$y")"
if [ "$y" -ne 0 ]; then
    echo "Quotient: $(expr "$x" / "$y")"
    echo "Remainder: $(expr "$x" % "$y")"
else
    echo "Division is not possible (division by zero)."
fi
```

**OUTPUT:**

```
Enter two integers:
18 6
Sum: 24
Difference: 12
Product: 108
Quotient: 3
Remainder: 0
```

**RESULT:** Thus, the shell script performed the arithmetic operations using the expr command.

# EXPERIMENT 1B

## SHELL SCRIPT – CHECK LEAP YEAR USING if-elif-else

**AIM:** To determine whether a given year is a leap year using if-elif-else statements in a shell script.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, Terminal, Bash shell.

### PROCEDURE:

1. Open the Fedora terminal and create leapyear.sh.

2. Type the program, save it, and make it executable with chmod +x leapyear.sh.

3. Run ./leapyear.sh and enter a year.

### PROGRAM:

```
echo "Enter a year:"
read yr
if [ $((yr % 400)) -eq 0 ]; then
    echo "$yr is a leap year."
elif [ $((yr % 100)) -eq 0 ]; then
    echo "$yr is not a leap year."
elif [ $((yr % 4)) -eq 0 ]; then
    echo "$yr is a leap year."
else
    echo "$yr is not a leap year."
fi
```

### OUTPUT:

```
Enter a year:
2028
2028 is a leap year.
```

**RESULT:** Thus, the script correctly checked whether the entered year was a leap year.

# EXPERIMENT 2A

## SHELL SCRIPT – REVERSE A NUMBER USING while LOOP

**AIM:** To reverse the digits of an integer using a while loop in a shell script.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, Terminal, Bash shell.

### PROCEDURE:

1. Create reverse.sh in the Fedora terminal.

2. Enter and save the script.

3. Run chmod +x reverse.sh followed by ./reverse.sh.

4. Enter a non-negative integer and observe the reversed number.

### PROGRAM:

```
echo "Enter a number:"
read n
orig=$n
reverse=0
while [ "$n" -gt 0 ]; do
    rem=$(expr "$n" % 10)
    reverse=$(expr "$reverse" \* 10 + "$rem")
    n=$(expr "$n" / 10)
done
echo "Reverse of $orig is: $reverse"
```

### OUTPUT:

```
Enter a number:
4567
Reverse of 4567 is: 7654
```

**RESULT:** Thus, the script correctly reversed the digits of the entered number using a while loop.

# EXPERIMENT 2B

## SHELL SCRIPT – FIBONACCI SERIES USING for LOOP

**AIM:** To generate the Fibonacci series for a specified number of terms using a for loop.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, Terminal, Bash shell.

### PROCEDURE:

1. Create fibonacci.sh.
2. Enter the program and save the file.
3. Make the file executable and run it.
4. Enter the required number of terms.

### PROGRAM:

```
echo "Enter number of terms:"
read terms
first=0
second=1
echo "Fibonacci series:"
for ((k=1; k<=terms; k++)); do
    echo -n "$first "
    sum=$((first + second))
    first=$second
    second=$sum
done
echo
```

### OUTPUT:

```
Enter number of terms:
10
Fibonacci series:
0 1 1 2 3 5 8 13 21 34
```

**RESULT:** Thus, the Fibonacci series was generated successfully using a for loop.

# EXPERIMENT 3

## SYSTEM CALLS – fork(), exec(), getpid(), opendir(), readdir()

**AIM:** To demonstrate process creation, process identification, program execution and directory handling using system/library calls in C.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

**PROCEDURE:**

1. Start.
2. Include the header files stdio.h, stdlib.h, unistd.h, sys/types.h, sys/wait.h and dirent.h.
3. Declare a variable of type pid_t to store the return value of fork().
4. Call fork() to create a child process.
5. If fork() returns a negative value, process creation has failed; display an error and stop.
6. If fork() returns 0, the current process is the child: display its PID using getpid() and its parent's PID using getppid(), then replace its image with 'ls -l' using execlp().
7. If fork() returns a positive value, the current process is the parent: display its PID and the child's PID, then wait for the child to finish using wait().
8. In the parent, open the current directory using opendir() and list every entry using readdir().
9. Close the directory using closedir().
10. Stop.

**PROGRAM:**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <dirent.h>

int main(void)
{
    pid_t child = fork();

    if (child < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child == 0)
    {
        printf("Child  : PID = %d, Parent PID = %d\n", getpid(), getppid());
        printf("Child  : running 'ls -l' using execlp...\n");
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec failed");
        exit(1);
    }
    else
    {
        printf("Parent : PID = %d, Child PID = %d\n", getpid(), child);
        fflush(stdout);
        wait(NULL);
```

```
        DIR *dp = opendir(".");
        if (dp == NULL)
        {
            perror("opendir failed");
            return 1;
        }
        printf("\nDirectory entries (readdir):\n");
        struct dirent *ent;
        while ((ent = readdir(dp)) != NULL)
            printf("%s\n", ent->d_name);
        closedir(dp);
    }
    return 0;
}
```

**OUTPUT:**

```
Parent : PID = 190, Child PID = 191
Child  : PID = 191, Parent PID = 190
Child  : running 'ls -l' using execlp...
total 32
drwxr-xr-x 2 root root  4096 Oct 29 15:30 Documents
-rw-r--r-- 1 root root     1 Oct 29 15:30 notes.txt
-rwxr-xr-x 1 root root 16512 Oct 29 15:30 system_calls
-rw-r--r-- 1 root root     1 Oct 29 15:30 system_calls.c

Directory entries (readdir):
system_calls.c
Documents
.
system_calls
notes.txt
..
```

**RESULT:** Thus, the fork(), execlp(), getpid(), getppid(), opendir() and readdir() functions were implemented successfully to demonstrate process creation, program execution and directory handling.

# EXPERIMENT 4

## CPU SCHEDULING – FCFS, SJF, PRIORITY, ROUND ROBIN

**AIM:** To implement the CPU scheduling algorithms First Come First Serve (FCFS), Shortest Job First (SJF), Priority Scheduling and Round Robin, and to calculate the waiting time and turnaround time of the processes.

**PROCEDURE:**

1. Start the Fedora operating system using Oracle VirtualBox.
2. Open the terminal and create a Python program file.
3. Write the programs for the given CPU scheduling algorithms.
4. Execute each program using the Python 3 interpreter.
5. Enter the required process details such as burst time, priority and time quantum.
6. Calculate and display the waiting time and turnaround time of each process.
7. Calculate the average waiting time and average turnaround time.
8. Verify the output and record the results.

**PROGRAM:**

**(A) First Come First Serve (FCFS)**

```
n = int(input("Enter the number of processes: "))
print("Enter the burst times:")
bt = list(map(int, input().split()))

wt = [0] * n
tat = [0] * n
for i in range(1, n):
    wt[i] = wt[i - 1] + bt[i - 1]
for i in range(n):
    tat[i] = wt[i] + bt[i]

print("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time")
for i in range(n):
    print(f"P{i + 1}\t{bt[i]}\t\t{wt[i]}\t\t{tat[i]}")

print("\nAverage Waiting Time    :", round(sum(wt) / n, 2))
print("Average Turnaround Time :", round(sum(tat) / n, 2))
```

**(B) Shortest Job First (SJF)**

```
n = int(input("Enter the number of processes: "))
print("Enter the burst times:")
bt = list(map(int, input().split()))

order = sorted(range(n), key=lambda i: bt[i])   # shortest burst first
wt = [0] * n
tat = [0] * n
elapsed = 0
for i in order:
    wt[i] = elapsed
    elapsed += bt[i]
    tat[i] = elapsed

print("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time")
for i in order:
    print(f"P{i + 1}\t{bt[i]}\t\t{wt[i]}\t\t{tat[i]}")
```

```
print("\nAverage Waiting Time    :", round(sum(wt) / n, 2))
print("Average Turnaround Time :", round(sum(tat) / n, 2))
```

### (C) Priority Scheduling

```
n = int(input("Enter the number of processes: "))
proc = []
for i in range(n):
    b = int(input(f"Burst time of P{i + 1}: "))
    p = int(input(f"Priority of P{i + 1}: "))
    proc.append((i + 1, b, p))

# smaller number = higher priority
proc.sort(key=lambda t: t[2])

elapsed = 0
total_wt = total_tat = 0
print("\nProcess\tBurst\tPriority\tWaiting\tTurnaround")
for pid, b, p in proc:
    wait = elapsed
    elapsed += b
    total_wt += wait
    total_tat += elapsed
    print(f"P{pid}\t{b}\t{p}\t\t{wait}\t{elapsed}")

print("\nAverage Waiting Time    :", round(total_wt / n, 2))
print("Average Turnaround Time :", round(total_tat / n, 2))
```

### (D) Round Robin

```
n = int(input("Enter the number of processes: "))
bt = list(map(int, input("Enter the burst times: ").split()))
q = int(input("Enter the time quantum: "))

remaining = bt[:]
wt = [0] * n
tat = [0] * n
clock = 0

while any(r > 0 for r in remaining):
    for i in range(n):
        if remaining[i] == 0:
            continue
        run = min(q, remaining[i])
        clock += run
        remaining[i] -= run
        if remaining[i] == 0:
            tat[i] = clock
            wt[i] = clock - bt[i]

print("\nProcess\tBurst\tWaiting\tTurnaround")
for i in range(n):
    print(f"P{i + 1}\t{bt[i]}\t{wt[i]}\t{tat[i]}")

print("\nAverage Waiting Time    :", round(sum(wt) / n, 2))
print("Average Turnaround Time :", round(sum(tat) / n, 2))
```

## OUTPUT:

### (A) FCFS

```
Enter the number of processes: 4
Enter the burst times:
8 4 9 5

Process Burst Time      Waiting Time    Turnaround Time
P1      8               0               8
P2      4               8               12
P3      9               12              21
```

```
P4      5               21              26

Average Waiting Time    : 10.25
Average Turnaround Time : 16.75
```

### (B) SJF

```
Enter the number of processes: 4
Enter the burst times:
8 4 9 5

Process Burst Time      Waiting Time    Turnaround Time
P2      4               0               4
P4      5               4               9
P1      8               9               17
P3      9               17              26

Average Waiting Time    : 7.5
Average Turnaround Time : 14.0
```

### (C) Priority

```
Enter the number of processes: 4
Burst time of P1: 8
Priority of P1: 2
Burst time of P2: 4
Priority of P2: 1
Burst time of P3: 9
Priority of P3: 4
Burst time of P4: 5
Priority of P4: 3

Process Burst   Priority        Waiting Turnaround
P2      4       1               0       4
P1      8       2               4       12
P4      5       3               12      17
P3      9       4               17      26

Average Waiting Time    : 8.25
Average Turnaround Time : 14.75
```

### (D) Round Robin

```
Enter the number of processes: 4
Enter the burst times: 8 4 9 5
Enter the time quantum: 3

Process Burst   Waiting Turnaround
P1      8       15      23
P2      4       12      16
P3      9       17      26
P4      5       16      21

Average Waiting Time    : 15.0
Average Turnaround Time : 21.5
```

**RESULT:** Thus, the CPU scheduling algorithms FCFS, SJF, Priority Scheduling and Round Robin were implemented and executed successfully, and the waiting time, turnaround time and their averages were calculated.

# EXPERIMENT 5

## IPC USING SHARED MEMORY

**AIM:** To write a C program to perform Inter Process Communication (IPC) using shared memory between a sender process and a receiver process.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

### ALGORITHM:

1. SENDER: Define the key and the size of the shared memory segment.
2. SENDER: Allocate the shared memory segment using shmget().
3. SENDER: Attach the segment to the process using shmat().
4. SENDER: Read a message from the user and store it in the shared memory.
5. SENDER: Detach the segment using shmdt().
6. RECEIVER: Obtain the same shared memory segment using shmget() with the same key.
7. RECEIVER: Attach the segment using shmat().
8. RECEIVER: Print the message written by the sender.
9. RECEIVER: Detach the segment using shmdt() and remove it using shmctl() with IPC_RMID.

### PROGRAM:

#### (A) sender.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_KEY  5678
#define SHM_SIZE 1024

int main()
{
    int id;
    char *mem;

    id = shmget(SHM_KEY, SHM_SIZE, 0666 | IPC_CREAT);
    if (id == -1)
    {
        perror("shmget");
        return 1;
    }

    mem = (char *)shmat(id, NULL, 0);
    if (mem == (char *)-1)
    {
        perror("shmat");
        return 1;
    }

    printf("Sender   : enter the message: ");
    fgets(mem, SHM_SIZE, stdin);
    printf("Sender   : message written to shared memory -> %s", mem);

    shmdt(mem);
```

```
    return 0;
}
```

### (B) receiver.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_KEY  5678
#define SHM_SIZE 1024

int main()
{
    int id;
    char *mem;

    id = shmget(SHM_KEY, SHM_SIZE, 0666);
    if (id == -1)
    {
        perror("shmget");
        return 1;
    }

    mem = (char *)shmat(id, NULL, 0);
    if (mem == (char *)-1)
    {
        perror("shmat");
        return 1;
    }

    printf("Receiver : message read from shared memory -> %s", mem);

    shmdt(mem);
    shmctl(id, IPC_RMID, NULL);
    return 0;
}
```

### OUTPUT:

```
Sender   : enter the message: Hello from the Sender process
Sender   : message written to shared memory -> Hello from the Sender process
Receiver : message read from shared memory -> Hello from the Sender process
```

**RESULT:** Thus, Inter-Process Communication using shared memory was implemented and executed successfully in Fedora Linux.

# EXPERIMENT 6

## PRODUCER–CONSUMER PROBLEM USING SEMAPHORES

**AIM:** To implement the Producer–Consumer problem using semaphores for process synchronization in the Linux operating system.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

**PROCEDURE:**

1. Start the Fedora operating system using Oracle VirtualBox.
2. Open the terminal and create a C program.
3. Declare the semaphores mutex, full and empty (buffer size = 4) and define the wait (down) and signal (up) operations.
4. Producer: if the buffer is not full and the mutex is free, produce an item and place it in the buffer.
5. Consumer: if the buffer is not empty and the mutex is free, remove an item from the buffer.
6. The empty and full semaphores prevent buffer overflow and underflow, and the mutex protects access to the buffer.
7. Display the produced and consumed items.
8. Compile, execute and verify the output.

**PROGRAM:**

```
#include <stdio.h>
#include <stdlib.h>

int mutex = 1, full = 0, empty = 4, item = 0;

void producer();
void consumer();
int down(int);
int up(int);

int main()
{
    int ch;
    printf("\n1.Producer\n2.Consumer\n3.Exit");
    while (1)
    {
        printf("\nEnter your choice: ");
        scanf("%d", &ch);
        switch (ch)
        {
        case 1:
            if (mutex == 1 && empty != 0)
                producer();
            else
                printf("Buffer is full!!");
            break;
        case 2:
            if (mutex == 1 && full != 0)
                consumer();
            else
                printf("Buffer is empty!!");
            break;
        case 3:
            exit(0);
```

```
        default:
            printf("Invalid choice!!");
        }
    }
    return 0;
}

int down(int s) { return --s; }
int up(int s)   { return ++s; }

void producer()
{
    mutex = down(mutex);
    full  = up(full);
    empty = down(empty);
    item++;
    printf("Producer produced item %d", item);
    mutex = up(mutex);
}

void consumer()
{
    mutex = down(mutex);
    full  = down(full);
    empty = up(empty);
    printf("Consumer consumed item %d", item);
    item--;
    mutex = up(mutex);
}
```

**OUTPUT:**

```
1.Producer
2.Consumer
3.Exit
Enter your choice: 1
Producer produced item 1
Enter your choice: 1
Producer produced item 2
Enter your choice: 1
Producer produced item 3
Enter your choice: 1
Producer produced item 4
Enter your choice: 1
Buffer is full!!
Enter your choice: 2
Consumer consumed item 4
Enter your choice: 2
Consumer consumed item 3
Enter your choice: 2
Consumer consumed item 2
Enter your choice: 2
Consumer consumed item 1
Enter your choice: 2
Buffer is empty!!
Enter your choice: 3
```

**RESULT:** Thus, the Producer–Consumer problem was implemented and executed successfully using simulated semaphore operations in C.

# EXPERIMENT 7

## BANKER'S ALGORITHM – DEADLOCK AVOIDANCE

**AIM:** To implement the Banker's Algorithm for deadlock avoidance and determine whether the system is in a safe state.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

**PROCEDURE:**

1. Calculate the Need matrix: Need = Max – Allocation.

2. Initialize Work = Available and Finish[i] = false for all i.

3. Find a process i such that Finish[i] = false and Need[i] <= Work.

4. If no such i exists, go to step 6.

5. Set Work = Work + Allocation[i], Finish[i] = true and go to step 3.

6. If Finish[i] = true for all i, the system is in a safe state; print the safe sequence.

7. Otherwise, the system is not in a safe state.

**PROGRAM:**

```
#include <stdio.h>

#define P 5
#define R 3

int main()
{
    int alloc[P][R] = {
        {1, 1, 0},
        {2, 1, 2},
        {3, 0, 1},
        {2, 1, 1},
        {0, 0, 2}
    };
    int max[P][R] = {
        {5, 3, 2},
        {3, 2, 2},
        {9, 0, 2},
        {2, 2, 2},
        {4, 3, 3}
    };
    int avail[R] = {3, 2, 2};

    int need[P][R], done[P] = {0}, seq[P], cnt = 0;
    int i, j, pass;

    for (i = 0; i < P; i++)
        for (j = 0; j < R; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    printf("Need matrix:\n");
    for (i = 0; i < P; i++)
    {
        printf("P%d: ", i);
        for (j = 0; j < R; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
```

```
    for (pass = 0; pass < P; pass++)
    {
        for (i = 0; i < P; i++)
        {
            if (done[i])
                continue;
            int ok = 1;
            for (j = 0; j < R; j++)
                if (need[i][j] > avail[j])
                {
                    ok = 0;
                    break;
                }
            if (ok)
            {
                for (j = 0; j < R; j++)
                    avail[j] += alloc[i][j];
                seq[cnt++] = i;
                done[i] = 1;
            }
        }
    }

    if (cnt == P)
    {
        printf("\nThe system is in a SAFE state.\nSafe sequence: ");
        for (i = 0; i < P; i++)
            printf("P%d%s", seq[i], i < P - 1 ? " -> " : "\n");
    }
    else
        printf("\nThe system is NOT in a safe state (deadlock possible).\n");

    return 0;
}
```

**OUTPUT:**

```
Need matrix:
P0: 4 2 2
P1: 1 1 0
P2: 6 0 1
P3: 0 1 1
P4: 4 3 1

The system is in a SAFE state.
Safe sequence: P1 -> P3 -> P4 -> P0 -> P2
```

**RESULT:** Thus, the Banker's Algorithm for deadlock avoidance was implemented successfully, and the system was found to be in a safe state.

# EXPERIMENT 8

## CONTIGUOUS MEMORY ALLOCATION – BEST FIT

**AIM:** To implement the Best Fit memory allocation algorithm and allocate memory blocks to processes efficiently.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

### PROCEDURE:

1. Start the Fedora operating system using Oracle VirtualBox.
2. Open the terminal and create a C program.
3. Enter the number of memory blocks and processes.
4. Read the sizes of the memory blocks and the processes.
5. For each process, find the smallest available block that can accommodate it.
6. Allocate the process to that block and reduce the block's remaining size.
7. Repeat until all processes have been considered.
8. Display the allocation details and verify the output.

### PROGRAM:

```
#include <stdio.h>

int main()
{
    int blk[20], proc[20], alloc[20];
    int nb, np, i, j, best;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter the block sizes:\n");
    for (i = 0; i < nb; i++)
        scanf("%d", &blk[i]);

    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter the process sizes:\n");
    for (i = 0; i < np; i++)
        scanf("%d", &proc[i]);

    for (i = 0; i < np; i++)
    {
        alloc[i] = -1;
        best = -1;
        for (j = 0; j < nb; j++)
        {
            if (blk[j] >= proc[i] && (best == -1 || blk[j] < blk[best]))
                best = j;
        }
        if (best != -1)
        {
            alloc[i] = best;
            blk[best] -= proc[i];
        }
    }

    printf("\nProcess\tSize\tBlock Allocated\n");
    for (i = 0; i < np; i++)
```

```
    {
        printf("P%d\t%d\t", i + 1, proc[i]);
        if (alloc[i] != -1)
            printf("Block %d\n", alloc[i] + 1);
        else
            printf("Not Allocated\n");
    }
    return 0;
}
```

**OUTPUT:**

```
Enter number of memory blocks: 5
Enter the block sizes:
150 450 250 350 550
Enter number of processes: 4
Enter the process sizes:
200 400 120 480

Process Size    Block Allocated
P1      200     Block 3
P2      400     Block 2
P3      120     Block 1
P4      480     Block 5
```

**RESULT:** Thus, the Best Fit memory allocation algorithm was implemented and executed successfully in Fedora Linux, and memory blocks were allocated to the processes according to their requirements.

# EXPERIMENT 9

## CONTIGUOUS MEMORY ALLOCATION – FIRST FIT

**AIM:** To implement the First Fit memory allocation algorithm in C.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

### PROCEDURE:

1. Start Fedora Linux using Oracle VirtualBox.
2. Open the terminal and create a C program.
3. Enter the number of memory blocks and processes.
4. Read the memory block sizes and process sizes.
5. For every process, scan the memory blocks sequentially.
6. Allocate the process to the first block having sufficient space.
7. Update the remaining size of the allocated block.
8. Display the memory allocation details.
9. Stop.

### PROGRAM:

```
#include <stdio.h>

int main()
{
    int blk[20], proc[20], alloc[20];
    int nb, np, i, j;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter the block sizes:\n");
    for (i = 0; i < nb; i++)
        scanf("%d", &blk[i]);

    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter the process sizes:\n");
    for (i = 0; i < np; i++)
        scanf("%d", &proc[i]);

    for (i = 0; i < np; i++)
    {
        alloc[i] = -1;
        for (j = 0; j < nb; j++)
        {
            if (blk[j] >= proc[i])
            {
                alloc[i] = j;
                blk[j] -= proc[i];
                break;
            }
        }
    }

    printf("\nProcess\tSize\tBlock Allocated\n");
    for (i = 0; i < np; i++)
    {
        printf("P%d\t%d\t", i + 1, proc[i]);
```

```
        if (alloc[i] == -1)
            printf("Not Allocated\n");
        else
            printf("Block %d\n", alloc[i] + 1);
    }
    return 0;
}
```

**OUTPUT:**

```
Enter number of memory blocks: 5
Enter the block sizes:
150 450 250 350 550
Enter number of processes: 4
Enter the process sizes:
200 400 120 480

Process Size    Block Allocated
P1      200     Block 2
P2      400     Block 5
P3      120     Block 1
P4      480     Not Allocated
```

**RESULT:** Thus, the First Fit memory allocation algorithm was implemented and executed successfully using C in Fedora Linux.

# EXPERIMENT 10

## PAGE REPLACEMENT – FIFO

**AIM:** To implement the First In First Out (FIFO) page replacement algorithm and calculate the total number of page faults.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, Python 3, Terminal.

### PROCEDURE:

1. Read the length of the reference string and the pages in it.
2. Read the number of page frames and initialise all frames as empty.
3. For each page, check whether it is already present in a frame.
4. If it is present, there is no page fault.
5. If it is absent, replace the page that entered memory first (oldest page) and increase the page-fault count.
6. Display the frame contents after every reference.
7. Display the total number of page faults.

### PROGRAM:

```
def fifo(refs, nframes):
    frames = [-1] * nframes
    ptr = 0
    faults = 0
    for pg in refs:
        if pg in frames:
            print(f"{pg} -> No Page Fault")
        else:
            frames[ptr] = pg
            ptr = (ptr + 1) % nframes
            faults += 1
            shown = " ".join(str(f) if f != -1 else "-" for f in frames)
            print(f"{pg} -> {shown}")
    return faults


size = int(input("Enter the size of the reference string: "))
refs = []
for i in range(size):
    refs.append(int(input(f"Enter [{i + 1:2}] : ")))
nframes = int(input("Enter the number of page frames: "))
print()
total = fifo(refs, nframes)
print("\nTotal page faults:", total)
```

### OUTPUT:

```
Enter the size of the reference string: 10
Enter [ 1] : 1
Enter [ 2] : 2
Enter [ 3] : 3
Enter [ 4] : 4
Enter [ 5] : 1
Enter [ 6] : 2
Enter [ 7] : 5
Enter [ 8] : 1
Enter [ 9] : 2
Enter [10] : 3
```

```
Enter the number of page frames: 3

1 -> 1 - -
2 -> 1 2 -
3 -> 1 2 3
4 -> 4 2 3
1 -> 4 1 3
2 -> 4 1 2
5 -> 5 1 2
1 -> No Page Fault
2 -> No Page Fault
3 -> 5 3 2

Total page faults: 8
```

**RESULT:** Thus, the FIFO page replacement algorithm was implemented and executed successfully using Python in Fedora Linux, and the total number of page faults was calculated.

# EXPERIMENT 11

## PAGE REPLACEMENT – LRU

**AIM:** To implement the Least Recently Used (LRU) page replacement algorithm and calculate the total number of page faults.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

**PROCEDURE:**

1. Start the process.
2. Read the number of frames, the number of pages and the reference string.
3. Keep a time stamp (counter value) for every frame, updated whenever the page in it is used.
4. If the page is already in a frame (hit), update its time stamp.
5. If it is not (fault), use an empty frame if available; otherwise replace the page with the smallest time stamp (least recently used).
6. Display the frames after every reference.
7. Display the total number of page faults.
8. Stop the process.

**PROGRAM:**

```
#include <stdio.h>

int main()
{
    int frames[10], pages[30], stamp[10];
    int n, m, i, j, victim, faults = 0, clock = 0;
    int hit, oldest;

    printf("Enter number of frames: ");
    scanf("%d", &m);
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    for (i = 0; i < m; i++)
    {
        frames[i] = -1;
        stamp[i] = 0;
    }

    printf("\nPage\tFrames\n");
    for (i = 0; i < n; i++)
    {
        hit = 0;
        for (j = 0; j < m; j++)
        {
            if (frames[j] == pages[i])
            {
                hit = 1;
                stamp[j] = ++clock;
                break;
            }
        }
```

```
        if (!hit)
        {
            faults++;
            victim = -1;
            for (j = 0; j < m; j++)
                if (frames[j] == -1)
                {
                    victim = j;
                    break;
                }
            if (victim == -1)
            {
                victim = 0;
                oldest = stamp[0];
                for (j = 1; j < m; j++)
                    if (stamp[j] < oldest)
                    {
                        oldest = stamp[j];
                        victim = j;
                    }
            }
            frames[victim] = pages[i];
            stamp[victim] = ++clock;
        }

        printf("%d\t", pages[i]);
        for (j = 0; j < m; j++)
        {
            if (frames[j] == -1)
                printf("-\t");
            else
                printf("%d\t", frames[j]);
        }
        printf("%s\n", hit ? "(hit)" : "(fault)");
    }

    printf("\nTotal Page Faults = %d\n", faults);
    return 0;
}
```

**OUTPUT:**

```
Enter number of frames: 3
Enter number of pages: 12
Enter reference string: 1 2 3 4 1 2 5 1 2 3 4 5

Page    Frames
1       1       -       -       (fault)
2       1       2       -       (fault)
3       1       2       3       (fault)
4       4       2       3       (fault)
1       4       1       3       (fault)
2       4       1       2       (fault)
5       5       1       2       (fault)
1       5       1       2       (hit)
2       5       1       2       (hit)
3       3       1       2       (fault)
4       3       4       2       (fault)
5       3       4       5       (fault)

Total Page Faults = 10
```

**RESULT:** Thus, the Least Recently Used (LRU) page replacement algorithm was implemented successfully using C in Fedora Linux, and the total number of page faults was calculated.

# EXPERIMENT 12

## PAGE REPLACEMENT – OPTIMAL

**AIM:** To implement the Optimal page replacement algorithm in C and calculate the total number of page faults.

**SOFTWARE REQUIRED:** Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

**PROCEDURE:**

1. Start the process.
2. Read the number of frames, the number of pages and the reference string.
3. If the page is already in a frame (hit), do nothing.
4. If it is not (fault), use an empty frame if available.
5. Otherwise, replace the page whose next use is farthest in the future (or which is never used again).
6. Display the frames after every reference.
7. Display the total number of page faults.
8. Stop the process.

**PROGRAM:**

```
#include <stdio.h>

int main()
{
    int frames[10], pages[30];
    int n, m, i, j, k, victim, faults = 0;
    int hit, farthest, next;

    printf("Enter number of frames: ");
    scanf("%d", &m);
    printf("Enter number of pages: ");
    scanf("%d", &n);
    if (m < 1 || m > 10 || n < 1 || n > 30)
    {
        printf("Invalid input size\n");
        return 1;
    }
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    for (i = 0; i < m; i++)
        frames[i] = -1;

    printf("\nPage\tFrames\n");
    for (i = 0; i < n; i++)
    {
        hit = 0;
        for (j = 0; j < m; j++)
            if (frames[j] == pages[i])
            {
                hit = 1;
                break;
            }

        if (!hit)
        {
```

```
            faults++;
            victim = -1;
            for (j = 0; j < m; j++)
                if (frames[j] == -1)
                {
                    victim = j;
                    break;
                }
            if (victim == -1)
            {
                farthest = -1;
                for (j = 0; j < m; j++)
                {
                    for (k = i + 1; k < n; k++)
                        if (frames[j] == pages[k])
                            break;
                    next = k;               /* n if never used again */
                    if (next > farthest)
                    {
                        farthest = next;
                        victim = j;
                    }
                }
            }
            frames[victim] = pages[i];
        }

        printf("%d\t", pages[i]);
        for (j = 0; j < m; j++)
        {
            if (frames[j] == -1)
                printf("-\t");
            else
                printf("%d\t", frames[j]);
        }
        printf("%s\n", hit ? "(hit)" : "(fault)");
    }

    printf("\nTotal Page Faults = %d\n", faults);
    return 0;
}
```

**OUTPUT:**

```
Enter number of frames: 3
Enter number of pages: 12
Enter reference string: 1 2 3 4 1 2 5 1 2 3 4 5

Page    Frames
1       1       -       -       (fault)
2       1       2       -       (fault)
3       1       2       3       (fault)
4       1       2       4       (fault)
1       1       2       4       (hit)
2       1       2       4       (hit)
5       1       2       5       (fault)
1       1       2       5       (hit)
2       1       2       5       (hit)
3       3       2       5       (fault)
4       4       2       5       (fault)
5       4       2       5       (hit)

Total Page Faults = 7
```

**RESULT:** Thus, the Optimal page replacement algorithm was implemented and executed successfully using C in Fedora Linux, and the total number of page faults was calculated.